

Intro	2
Cost Function	2
Binary Cross Entropy with Logits	2
The gradient	2
Evaluation Metrics	3
F1 macro score	3
Aggregation methods	4
FedAvg	4
Federated Proximal	5
Global vs local thresholds	5
Global:	6
Local:	6
Interpretation:	6
Base model	7
Dropout regularization	8
Batch normalization	8
Data generation	9
Data partitioning	9
Grid Search/Beta Sweep	10
MLP architecture (1 vs 4 networks vs branching)	11
1 network with 4 outputs, no branching	11
4 networks with 1 output each, no branching	11
1 Network with 4 outputs and branching	12
LASSO	13

Intro

The purpose of this document is to provide an overview of the theory behind the methods used in this project.

Please note:

This document is primarily for our own understanding of the theory, and as such does not necessarily provide theory on everything we have done, nor is all the theory fully complete. It also does not contain any results obtained in the course of the project, as that can be found in a separate document 'Results'. Furthermore, a lot of this document is related to work implemented before we took over the project, and is not relevant to the further work we have done.

First we go through theory relating to the MLP already implemented when we took over the project, hereunder the cost function and the primary evaluation metric we use. Thereafter we go through theory relating to our further work on the project. However most of the work we have done on the project can be found in the 'Results' document.

Cost Function

Binary Cross Entropy with Logits

<https://lucasdavid.github.io/blog/machine-learning/crossentropy-and-logits/>

The loss function we use to train the model is the binary cross entropy with logits.

In the output layer a sigmoid function is applied to each node, independently of each other.

For each of the four complications the binary cross entropy loss is

$$L_j = l_j \log(y_j) + (1 - l_j) \log(1 - y_j)$$

This is then averaged, so that we only have one loss function:

$$L = \frac{1}{4} \sum_{j=1}^4 L_j$$

Where y is the predicted probability of the complication and l is the true label $l \in \{0,1\}$.

For a batch this becomes:

$$L = \frac{1}{N} \sum_{n=1}^N \sum_{j=1}^4 L_{n,j}$$

With N being the batch size.

The gradient

The gradient of this is quite neat, it is simply the difference between the prediction and the target:

$$\nabla L = y_i - t_i$$

Evaluation Metrics

<https://towardsdatascience.com/micro-macro-weighted-averages-of-f1-score-clearly-explained-b603420b292f/>

The model is trained using binary cross-entropy with logits on the observed complication probability outcomes. During training, the model therefore learns to estimate the probability

$$P(\text{complication} \mid x)$$

for each patient based on the input features x .

The risk categories (low, mid, high) are not used during training. Instead, they are introduced only during evaluation by applying percentile thresholds (33rd and 67th percentiles) to the predicted probabilities.

This means that the model itself learns probability estimates, while the categorization is a post-training step used for evaluation.

The model is evaluated on one main metric, the F1 macro score, with the option of also looking at several other metrics.

We continue to look at the F1-score and the F1-macro-score throughout the project, so this is the most important metric.

F1 macro score

This is essentially a classification problem with 3 classes, low, mid, high and for each class we have the following values:

- TP : True Positive, a datapoint belongs to the given class, and was predicted to do so as well.

- FP : False Positive, a datapoint was predicted to belong to the given class, even though it does not.
- TN : True Negative, a datapoint does not belong to the given class, and was not predicted to.
- FN : False Negative, a datapoint was not predicted to belong to the given class, but it actually does.

Using these values we can now define the following values for each class:

Precision:

$$Precision_{class} = \frac{TP_{class}}{TP_{class} + FP_{class}}$$

In layman's terms this means: Of all the positive predictions I made, how many of them are truly positive?

Recall:

$$Recall_{class} = \frac{TP_{class}}{TP_{class} + FN_{class}}$$

In layman's terms this means: Of all the actual positive examples out there, how many of them did I correctly predict to be positive?

These values can then be used to define the F1-score for each class:

$$F1_{class} = 2 \cdot \frac{Precision_{class} \cdot Recall_{class}}{Precision_{class} + Recall_{class}}$$

The F1-macro score is now simply the average of this score:

$$F1_{macro} = \frac{F1_{low} + F1_{mid} + F1_{high}}{3}$$

Aggregation methods

FedAvg

The aggregation method in use when we took over the project was Federated Average. In this method, when the weights of each client is sent back to the central server, we compute a weighted average, using the number of patients of the client. This is a very basic and much

used method of aggregation. Throughout the project we have implemented other methods of aggregation, primarily FedProx, however, when testing these it turns out FedAvg is indeed the best one, and so we have continued to use that for the rest of the project.

Federated Proximal

FedProx (Federated Proximal) is an extension of the classical federated learning algorithm FedAvg. The purpose of FedProx is to handle heterogeneity between clients by penalizing client updates that deviate too much from the global model.

A common challenge in federated learning arises when the data across clients are non-IID. In such cases, model updates from different clients may point in different directions, which can lead to unstable training and poor convergence. This phenomenon is often referred to as client drift.

In FedAvg, each client k minimizes a local loss function f (binary cross-entropy with logits in our case):

$$\min_w f_k(w)$$

FedProx extends the classical federated learning algorithm FedAvg by adding a regularization term to the local optimization problem. The goal is to penalize clients where model-updates deviates a lot from the global model:

$$\min_w f_k(w) + \frac{\mu}{2} \|w - w^t\|^2$$

where

- $f_k(w)$ is the local loss function of client k
- w^t is the global model at iteration t
- μ is a hyperparameter controlling the strength of the regularization
- $\|w - w^t\|^2$ is the proximal term that penalizes large deviations from the global model

Global vs local thresholds

In our project complications are rare events, therefore direct evaluation on the binary outcomes is unstable. Instead, predicted probabilities are converted into three risk categories (low, mid and high). The categories are defined using the 33rd and 67th percentiles of the predicted probabilities. Two approaches can be used to compute these thresholds: global

thresholds and local thresholds. The choice between these approaches determines what we want the model to be good at.

Global:

With global thresholds, the percentiles are computed across all patients from all clinics. This means that the risk categories correspond to absolute risk levels in the overall population. A predicted probability therefore has the same interpretation across all clinics. Global thresholds evaluate

$$P(\text{complication} \mid x)$$

where x is a person's parameters (e.g age). This means we evaluate the model's ability to predict the probability for getting a complication given the inputs.

Local:

With local thresholds, percentiles are computed separately for each clinic. Each clinic therefore has its own risk thresholds. The categories represent the lowest, middle, and highest risk patients within that specific clinic. In this case, the macro F1-score evaluates whether the model correctly ranks patients within each clinic. A model can achieve a high score even if its absolute probability estimates are systematically biased, as long as it correctly orders patients by risk locally.

Interpretation:

The two evaluation strategies measure different aspects of model performance:

Global thresholds	Ability to estimate absolute risk across clinics
Local thresholds	Ability to rank patients within each clinic

When global thresholds are used, the same risk thresholds apply to all clinics. This means that a given predicted probability has the same interpretation everywhere. A patient predicted to have high risk will therefore be classified as high risk regardless of which clinic the patient belongs to.

In this setting, the evaluation measures whether the model can correctly identify patients with genuinely high complication risk across the entire population. The model must therefore learn

a risk function that works across clinics with different patient populations and complication rates.

Local thresholds work differently. Here, the risk thresholds are calculated separately for each clinic. As a result, the categories represent the lowest, middle, and highest risk patients within that clinic.

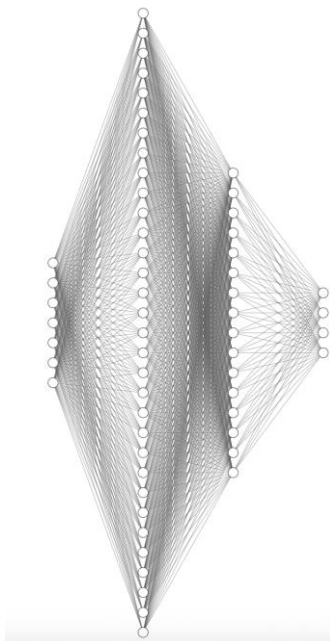
This means that the evaluation focuses on whether the model correctly ranks patients relative to others in the same clinic. A model can still perform well even if its predicted probabilities are slightly biased, as long as it correctly identifies which patients have higher risk than others locally.

Base model

The model in place upon our taking over the project was a MLP (Multilayer Perceptron) – so a neural network – implemented in PyTorch.

The model takes patient characteristics as input and predicts the probability of the four complications. The MLP consists of an input layer with 30 patient features after preprocessing, followed by two fully connected hidden layers with 128 and 64 neurons.

Thus the network can be illustrated as follows: (with the first 3 layers being scaled by $\frac{1}{4}$).



The two hidden layers use ReLU activation functions. Dropout regularization is applied between layers to reduce overfitting. The final layer has one output neuron for each complication, and the model outputs probabilities using a sigmoid activation function. During training, the model is optimized using binary cross-entropy with logits.

Dropout regularization

In dropout regularization we “turn off” a fraction of the neurons in each training step.

Concretely we are using a dropout probability of

$$p(dropout) = 0.2$$

This means that in a given training step, only about 80% of the neurons are active.

In the training data, there might be random correlations between features and noise. When the network is trained on this and is overfitted to the training data the neurons begin to *co-adapt*.

This means that the neurons are dependent on each other and only make precise predictions when the specific (random, noise-induced) correlations are presented in the data - which they might not be in the validation set. Because of this we introduce **dropout**. When we randomly turn off neurons in training steps we are forcing the neurons to work well independently - thus assisting in preventing overfitting.

When we use the network for inference (on the validation, test or real-world data) we again turn on all neurons. This will of course lead to too big predictions - since we are using 100% of the weights and not only 80% as we did in training. This is handled by actually scaling the “surviving” neurons with $\frac{1}{1-p}$ during training. So the network learns to have weights that are too low, because they will then be amplified later.

Worth noticing: Dropout introduces a tradeoff between

Not learning spurious interactive behaviour ↔ *Not learning ACTUAL interactive behaviour*

Too strong dropout can result in the model not learning complex interactions.

Batch normalization

Batch normalization does not directly help with overfitting, but helps more with just training in general, making it faster and more stable.

The batch normalization is applied in each hidden layer, so that the structure is

Linear combination → Batch normalization → Activation function (ReLU)

Batch normalization works by normalizing the activations of each neuron using the statistics of the current mini-batch. Suppose we have a batch of m samples and a hidden layer with k neurons. After the linear transformation, each neuron produces m activation values (one for

each sample in the batch). Batch normalization computes the mean and variance of these values for each neuron across the batch.

Data generation

The data we work with in the project is simulated. A previous part of the project has been to develop a code that generates the data. The datapoints correspond to patients that have undergone a third molar extraction surgery. The data generation process starts by generating people with demographics and characteristics that mimics those of the real world. Below is an excerpt of some of these variables:

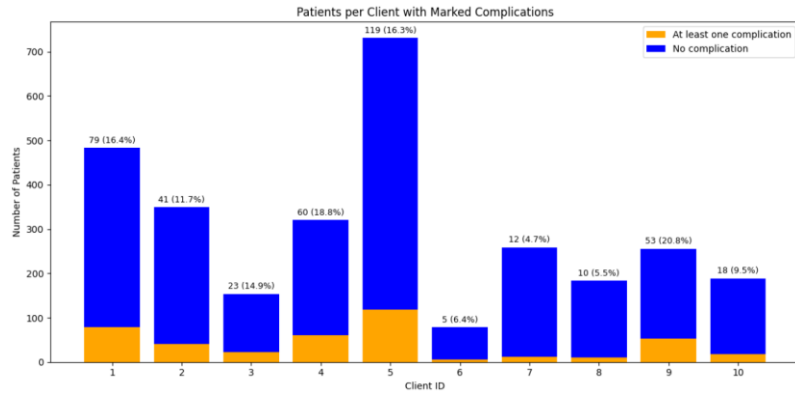
Variables	
Age	Impaction_Depth
Sex	Tooth_Angulation
Pain	Proximity_Nerve
Swelling	Root_Curvature
Pericoronitis	Bone_Density
Caries_Wisdom	Smoking

Table 1: Selected input variables used in the model.

Using knowledge from literature and clinical experts these characteristics are used to calculate a complication risk. The foundation of these calculations is that we have a base risk (the base incidence of the complication). We then multiply this base risk if a generated person has a characteristic that makes them more or less likely to get a complication. When the risk is calculated we draw from a uniform distribution in order to (randomly, but based on the probability) generate whether or not a simulated person gets the complication. This of course introduces stochasticity and noise in the data. Using the percentiles of the risk distribution we partition the data points into low, mid and high categories.

Data partitioning

From here data can be partitioned into clients to simulate the real-world scenario of having different dental clinics. This is done using a Dirichlet distribution. A parameter in the Dirichlet distribution is β , which controls how skewed the distribution is. Bigger values result in more skewed data and smaller values result in more uniformly distributed data. We have two different beta-parameters: One controlling distribution of data amount across clients and one controlling amount of positive cases (of complications) across clients. Below we have a figure illustrating this.



Here we see that the data is not evenly distributed across clients - neither in terms of number of datapoints or number of positive cases. This leads us to what we are specifically working on right now:

Grid Search/Beta Sweep

The idea behind implementing a Grid Search is to evaluate how robust the model is under different degrees of heterogeneity. Because we do not have access to real data we simulate a range of different realistic scenarios. This is done by systematically varying the parameters β_L and β_Q . Where high value of β_Q distribute the number of data points/patients uniformly across clients, likewise a high value of β_L distribute patients/data points with high-risk uniformly across clients.

By systematically varying these parameters, we generate datasets with different levels of heterogeneity. For each configuration we train both the local, centralized and federated model and evaluate the models using several metrics, but primarily F1-macro-score.

The purpose of this sweep is not to tune the model for a single dataset, but to understand how model performance changes under different realistic conditions. By evaluating performance across many heterogeneity settings, we can identify when federated learning performs similarly to centralized or local training. This provides insight into how the models might behave when deployed in the real-world.

MLP architecture (1 vs 4 networks vs branching)

In the last part of the project, we looked into different MLP architectures, to see if another architecture gives better results.

We have three main choices for the architecture of the network:

- 1 network with 4 outputs, no branching - this is the base MLP already implemented
- 4 networks with 1 output each, no branching
- 1 network with 4 outputs and branching - meaning that the last layers are not fully connected, and instead branches out into their 'own' network.

Below is a description of each as well as pros and cons.

1 network with 4 outputs, no branching

This is the network we have been working with so far. Here all the layers are fully connected.

Some reasons for choosing such an architecture are:

- If one feature affects several complications, this relation only needs to be learned once.
- Less risk of overfitting. The individual networks might become too specific in the task they are given.
- Less computationally heavy.
- Able to learn interactive relations in data.

4 networks with 1 output each, no branching

This completely separates the task and just trains 4 individual networks each outputting a probability between 0 and 1.

Pros of choosing this architecture:

- In the base MLP case there might be weights actively working against each other just to cancel out.
- Larger flexibility for the specific task in model architecture and training choices. Eg. some choices for loss calculation, learning rate ect. might be more beneficial for very rare vs less rare conditions.
- Better management of class imbalance. More common complications might influence the model a lot more, and the model may learn to ignore rare complications
- Easier to see how each model is performing ie. which complications are easiest to predict and which are more troublesome.

Cons:

- Heavy to compute
- might over fit

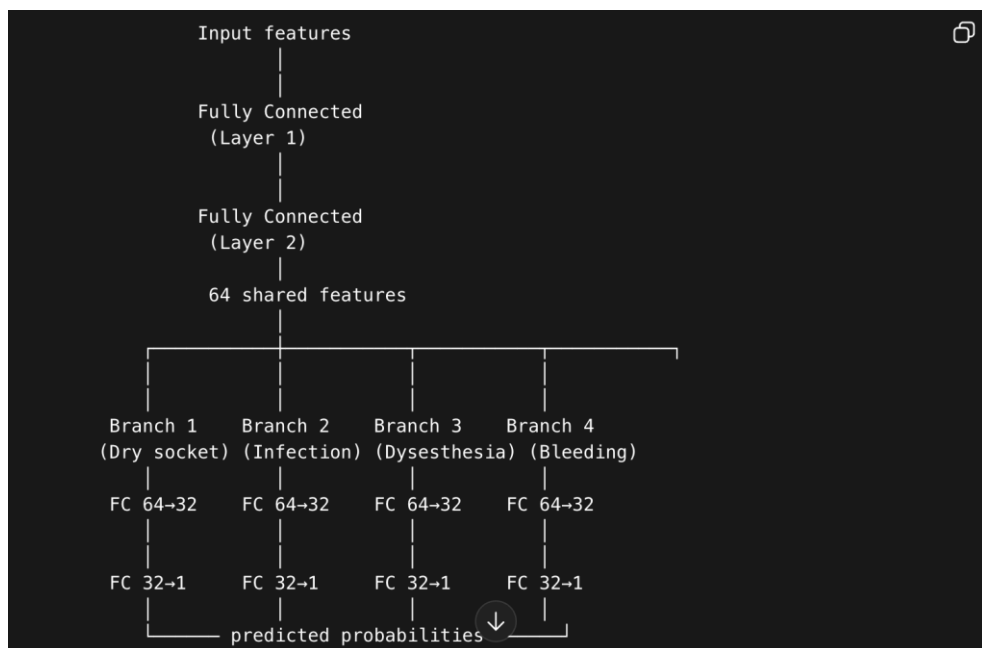
1 Network with 4 outputs and branching

This is a model architecture we explored as per Ruben's suggestion:

“Regardless, I would suggest a quick check of some different MLP architectural set-ups. Having a common backbone and only the output layer specifying four complications is probably suboptimal. You would probably want the MLP to branch out quite early so that each complication has its own dedicated layer(s). Please check if that's already the case”

- Mail from Ruben, 12/3.

In this architecture the first layers are shared in the model for all the complications. In one of the layers before the output layer, there is a branching so we have some neurons that are only associated with a specific condition. This is illustrated in the figure below:



This is a way to try to get the best of both worlds. It is not much heavier to train, and the features that actually affect the complications in the same can be learned in the same way. But we still have more flexibility in the last layer.

LASSO

During the Groundwork part of our project we used a method called Global LASSO. Most of the theory behind the methods used in the Groundwork part of the project can be found in either the individual Jupyter Notebook documents or in the Results file, however the LASSO is missing a bit, so we put it here:

LASSO is a way of fitting a model using linear regression, that also does feature selection. It does this by

$$\min_w \sum_{k=1}^K \left(\frac{1}{n_k} ||X^{(k)}w - y^{(k)}||^2 \right) - \lambda ||w||_1$$

The first expression is just a regular MSE.

The second term is the lasso term, where λ is a penalty: big $\lambda \rightarrow$ more feature selection, small $\lambda \rightarrow$ regular OLS.

This is a problem that is solved iteratively, where the weights of the model are initialized as 0.

$$w^{(0)} = 0$$

The server now sends these weights to each client. Each client then updates locally:

$$w^{(k)} = \arg \min_w \frac{1}{n_k} ||X^{(k)}w - y^{(k)}||^2 - \lambda ||w||_1$$

But they do not do this directly, instead they use **coordinate descent**.

First define for each feature j :

$$r_j^{(k)} = y^{(k)} - \sum_{i \neq j} X^{(k)}_i w_i$$

meaning ‘what is left if feature j is ignored?’.

Each coefficient is then updated using **soft thresholding**:

$$w_j = S\left(\frac{1}{n_k} X^{(k)} r_j^{(k)}, \lambda\right), \quad \text{where } S(z, \lambda) = \text{sign}(z) \cdot \max(|z| - \lambda, 0)$$

this is done for all j until convergence.

The weights are then sent back to the central server, and aggregated using FedAvg.